**UNIVERSIDAD TECNOLÓGICA NACIONAL**

**FACULTAD REGIONAL SANTA FE**

# Programación Competitiva

## Trabajo práctico

**Año 2024**


**Integrantes:**

- **Carthery Francisco**
- **Gonzalez Lautaro**
- **Marelli Octavio**

**Ingeniería en Sistemas de la Información - Comisión "A"**

**Profesores: Ambort, Daniel y Feresin, Mariano.**


**Fecha de entrega: 02/06/2024**

# Índice

1.0 Introducción ……... 2
2.0 Generación caso de prueba…… 2
3.0 Algoritmos …….. 3
3.1 Solución 1 ……...…… 3
3.2 Análisis teórico de complejidad ……...… 3
3.3 Análisis experimental de la complejidad……..... 4
3.4 Gráfica puntos en el plano ……...… 5
4.0 Divide y vencerás ……...…… 5
4.1 Karatsuba …….…… 6
4.2 Analisis teorico de complejidad ……... 8
4.3 Análisis experimental de la complejidad……. 8
4.4 Gráfica puntos en el plano ……... 9
5.0 Transformada rapida de Fourier ……. 9
5.2 Análisis teórico de complejidad ……..... 10
5.3 Análisis experimental de la complejidad…….…..……. 10
5.4 Gráfica puntos en el plano ……...…... 11
6.0 Bibliografía …….... 11

# 1. Introducción

En el presente trabajo práctico, nos enfocaremos en el análisis y la implementación de diversos algoritmos destinados a resolver la multiplicación de números enteros de 30 cifras o más. A lo largo de este trabajo, no solo implementaremos estos algoritmos, sino que también llevaremos a cabo mediciones experimentales utilizando entradas de diferentes tamaños y características. Esto nos permitirá obtener conclusiones fundamentadas sobre el rendimiento y la complejidad de cada enfoque, proporcionando una visión detallada de su aplicabilidad en diferentes contextos. Nuestro objetivo es identificar cuál de estos algoritmos ofrece un mejor desempeño en términos de tiempo de ejecución y recursos, lo cual es esencial para aplicaciones que requieren la manipulación de números extremadamente grandes.

---

# 2. Generación de casos de prueba

Para la prueba y medición de tiempos utilizamos entradas de distintos tamaños generadas de forma aleatoria. Las entradas generadas tienen un tamaño que va desde $2^0.100$ hasta $2^{10}.100$. Esto se puede observar en el funcionamiento del generador:

```
#include <bits/stdc++.h>

using namespace std;

int main(){

    ofstream archivo("input.in");
    srand(time(NULL));
    for(int j = 1; j < (1 << 11); j *= 2){
        string pal1, pal2;
        for(int i = 0; i < j * 100; i++){
            pal1 += char(rand() % 10 + '0'); //Entrada 1
            pal2 += char(rand() % 10 + '0'); //Entrada 2
        }
        archivo << pal1 << '\n' << pal2 << "\n\n";
    }

    return 0;
}
```

---

# 3. Algoritmos

## 3.1 Solución 1

Este algoritmo, de complejidad cuadrática, se basa en la multiplicación de dos números de forma clásica como se haría en papel. Inicialmente se multiplica el primer dígito del multiplicador por el multiplicando dígito a dígito, de derecha a izquierda, teniendo en cuenta el acarreo de posición.

Luego, este proceso se repite con el resto de los dígitos del multiplicador, realizando un desplazamiento conforme a la posición del dígito. Una vez finalizado este proceso, se suma este resultado parcial al resultado total de las iteraciones anteriores.

Para lograr esto, recibimos tanto multiplicador como multiplicando en dos variables de tipo string distintas, creamos un vector para el multiplicando y otro para el multiplicador y mediante un bucle pasamos dígito a dígito los valores de cada string a cada posición del vector correspondiente.

Una vez realizado esto, empezamos con el cálculo de la multiplicación mencionada anteriormente.

Implementación

## 3.2 Análisis teórico de complejidad

N: longitud de xpal (multiplicando)

M: longitud de ypal (multiplicador)

1. Cargar ambos vectores
    a. M + N
2. Multiplicación de cada dígito del multiplicador por el multiplicando
    a. N
3. Suma de resultado parcial y resultado total
    a. N+M
4. Repetición de pasos 2 y 3, M veces
    a. $M \cdot (2N + M)$
5. Mostrar resultado
    a. M + N

La complejidad final es la suma de paso 1, 4 y 5:

$M \cdot (2N + M) + 2N + 2M$

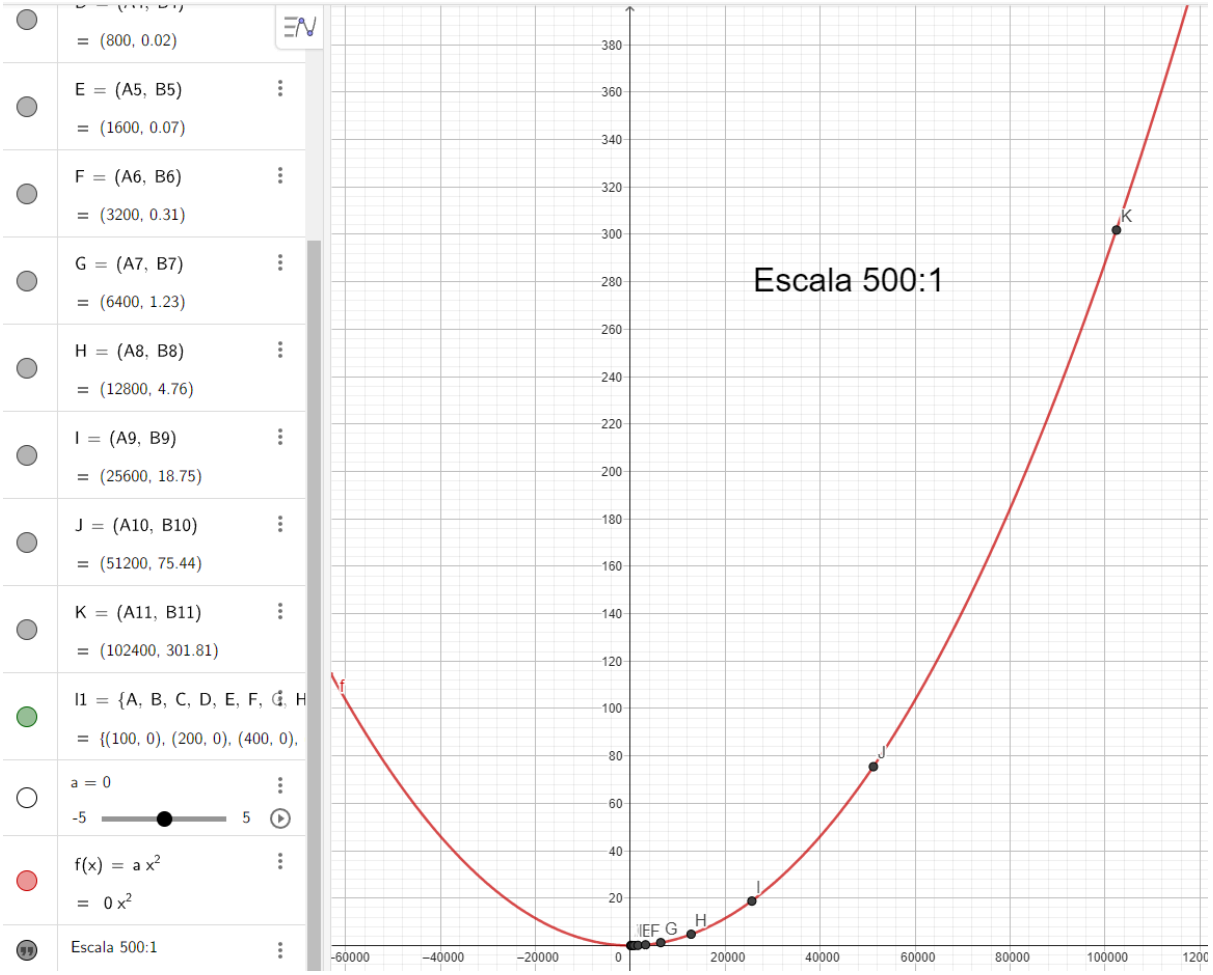
$2MN + M^2 + 2N + 2M$

Se puede apreciar que el término de mayor orden es cuadrático por lo que en notación O la complejidad del algoritmo es $O(n^2)$

### 3.3 Análisis experimental de la complejidad

| **Algoritmo 1 $O(n^2)$** | |
|---|---|
| Talla (N) | Tiempo[s] |
| 100 | 0.00034 |
| 200 | 0.00120 |
| 400 | 0.00472 |
| 800 | 0.01851 |
| 1600 | 0.07480 |
| 3200 | 0.30836 |
| 6400 | 1.23455 |
| 12800 | 4.75593 |
| 25600 | 18.7474 |
| 51200 | 75.4437 |
| 102400 | 301.813 |

### 3.4 Gráfica puntos en el plano:




Se puede apreciar que los tiempos obtenidos en función al tamaño de entrada adoptan la forma de una parábola(función cuadrática).

---

## 4. Divide y vencerás

Este tipo de algoritmos comprende uno de los más importantes paradigmas de diseño algorítmico. Este método está basado en la resolución recursiva de un problema dividiéndolo en dos o más subproblemas de igual tipo o similar. Esto se sigue realizando hasta que los problemas lleguen a ser lo suficientemente sencillos para resolverlos de forma directa. Dando así, lugar a la combinación de las soluciones de los subproblemas para dar la solución al problema original.

Este tipo de algoritmos es importante, ya que es la base de muchos algoritmos eficientes para resolver una gran cantidad de problemas. Entre ellos, ordenamiento (quicksort,

mergesort), multiplicación de números grandes (Karatsuba), análisis sintácticos, la transformada discreta de Fourier, búsqueda binaria, entre otros.

La resolución de problemas mediante esta técnica cuenta de los siguientes pasos:

1. Primero, debemos plantear el problema de forma tal que podamos descomponerlo en subproblemas del mismo tipo y menor tamaño.
2. En segundo lugar, se deben resolver todos los subproblemas recursivamente. El trabajar en problemas cada vez más chicos, nos permite tener casos bases los cuales serán resueltos de forma directa y acabarán con la recursión.
3. Por último, debemos combinar las soluciones obtenidas en el paso anterior para construir la solución original del problema.

La implementación a estos problemas se puede dar mediante un algoritmo recursivo, o mediante el uso de estructuras de datos (pila, cola, cola de prioridad) que almacenen las soluciones parciales. Esto permite el uso de técnicas como ramificación y acotación u optimización.

*Ventajas:*

1. Permite la resolución de problemas complejos.
2. Son algoritmos eficientes.
3. Al subdividir el problema, permite la ejecución en paralelo.
4. Hacen un uso eficiente de las memorias cachés.
5. Son más precisos en aritmética de punto flotante que una solución iterativa.

*Desventajas:*

1. Su principal desventaja es la lentitud en la repetición del proceso recursivo, ya que puede imprimir gastos indirectos en sus llamadas, junto con el almacenamiento en pila de las mismas.
2. Puede no ser aplicable cuando las interacciones no son predecibles o el problema general no se deriva de la suma directa y simple de los subproblemas, necesitando que las subsoluciones tengan interacciones y no se realicen de formas independientes

## 4.1 Karatsuba

El algoritmo de Karatsuba es una técnica eficiente para multiplicar números grandes. Se basa en la observación de que la multiplicación de dos números de *n* dígitos puede

dividirse en tres multiplicaciones de números más pequeños, junto con algunas operaciones de suma y resta. La idea principal detrás del algoritmo de Karatsuba es dividir los números en partes más pequeñas, multiplicarlas de forma recursiva y luego combinar los resultados de manera adecuada.

Dada la multiplicación de dos números grandes *x* e *y*, podemos expresar estos números como:

$$x = x_1 B^m + x_0$$

$$y = y_1 B^m + y_0$$

donde m es menor al tamaño de las cadenas *x* e *y*, y $x_1$, $x_0$, $y_1$, $y_0$ son partes de estos números divididos a la mitad y *B* es la base de los números a operar.

La multiplicación de *x* e *y* se puede expresar como:

$$xy = (x_1 B^m + x_0)(y_1 B^m + y_0)$$

$$= y_1 x_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0$$

Lo que Karatsuba descubrió es que la expresión $(x_1 y_0 + x_0 y_1)$ la cual consta de dos multiplicaciones se podía reexpresar de tal manera que solo conste de una sola multiplicación que no se haya realizado antes, la cual es $(x_1 + x_0)(y_0 + y_1) - x_1 y_1 - x_0 y_0$ .

El algoritmo de Karatsuba aprovecha esta expresión para reducir la cantidad de multiplicaciones necesarias. En lugar de realizar cuatro multiplicaciones (una para cada término), realiza tres multiplicaciones, que son las multiplicaciones de $x_1 y_1$ , $x_0 y_0$ y $(x_1 + x_0)(y_0 + y_1)$. Luego, combina los resultados de estas multiplicaciones intermedias adecuadamente para obtener el resultado final.

La implementación del algoritmo de Karatsuba generalmente se realiza de manera recursiva. Divide los números en partes más pequeñas hasta que sean lo suficientemente pequeños (por ejemplo, de un solo dígito), luego realiza las multiplicaciones intermedias y las combina para obtener el resultado final. La recursión se detiene cuando los números son lo suficientemente pequeños como para ser multiplicados de manera directa y eficiente.

En resumen, el algoritmo de Karatsuba es una técnica eficiente para multiplicar números grandes, aprovechando la división y la combinación de los números en partes más pequeñas para reducir la cantidad de multiplicaciones necesarias. Esto conduce a una mejora en la eficiencia computacional en comparación con los métodos de multiplicación estándar.

Implementación

## 4.2 Análisis teórico de la complejidad

Debido a que la implementación que nosotros llevamos a cabo es recursiva es más eficiente tomar a la variable *m* como *n/2* (n igual a la longitud de las cadenas) y la base utilizada es decimal, por lo tanto la fórmula antes mencionada queda de la siguiente manera:

$$x = x_1 10^{n/2} + x_0$$

$$y = y_1 10^{n/2} + y_0$$

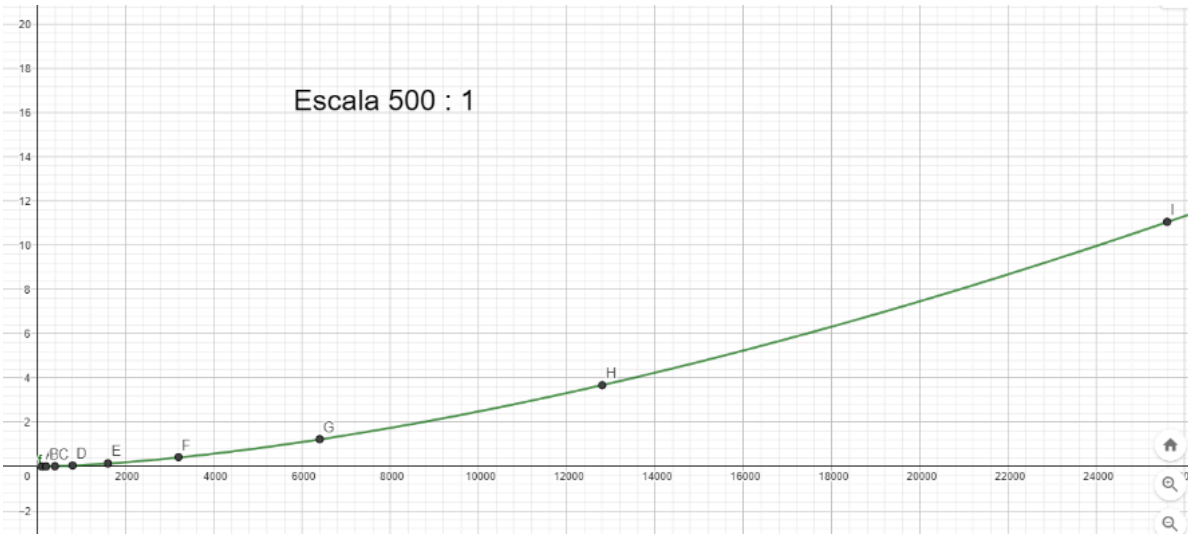
Luego, teniendo una constante k igual a la cantidad de veces que se dividirán la cadenas en 2, sabemos que se realizarán $3^k$ multiplicaciones de un solo dígito. Esto es lo mismo que decir $n^c$ donde $c = log_2 3$. Por lo tanto la complejidad del algoritmo Karatsuba en notación O es $O(n^{log2(3)})$.

## 4.3 Análisis experimental de la complejidad

| **Karatsuba $O(n^{log2(3)})$** | |
|---|---|
| Talla (N) | Tiempo[s] |
| 100 | 0.00184 |
| 200 | 0.00496 |
| 400 | 0.01507 |
| 800 | 0.04880 |
| 1600 | 0.14358 |
| 3200 | 0.42793 |
| 6400 | 1.29762 |

| | |
|---|---|
| 12800 | 3.68029 |
| 25600 | 11.0727 |
| 51200 | 33.1140 |
| 102400 | 99.4294 |

### 4.4 Gráfica puntos en el plano




## 5. Transformada rápida de Fourier

Una de las técnicas más rápidas y conocidas para la multiplicación de enteros grandes es el algoritmo de la transformada rápida de Fourier. Este es un algoritmo eficiente para computar la Transformada de Fourier Discreta (DFT) y su inversa. La DFT convierte una secuencia de valores, generalmente complejos, del dominio temporal al dominio frecuencial, permitiendo analizar las frecuencias que componen la señal original. Este algoritmo recibe dos vectores que representan los coeficientes de un polinomio y devuelve un vector que representa los coeficientes del polinomio resultante de la multiplicación de los dos primeros vectores.

La multiplicación de dos números grandes puede ser vista como la multiplicación de polinomios cuyos coeficientes son los dígitos de los números. La FFT se utiliza para transformar estos polinomios al dominio frecuencial, donde la multiplicación de polinomios se convierte en una multiplicación punto a punto de sus coeficientes transformados.

Posteriormente, se aplica la FFT inversa para volver al dominio temporal y obtener el resultado de la convolución, que corresponde al producto de los dos números originales.

Lo que devuelve el algoritmo es el resultado de la suma de los productos de los dígitos que se encuentran en una misma columna. Es decir, si se multiplican dos números A y B, siendo $A_nA_{n-1}\ldots A_i\ldots A_1A_0$ los dígitos de A y análogamente $B_mB_{m-1}\ldots B_j\ldots B_1B_0$ los dígitos de B. El resultado será R donde $R_hR_{h-1}\ldots R_k\ldots R_1R_0$, donde $R_k$ es igual a la suma de los términos $A_iB_j$ donde $i+j$ es igual a $k$.

Es por esto que cada elemento del vector que retorna el algoritmo puede tener más de un dígito por lo que para poder almacenar el número en un vector donde cada posición representa un solo dígito del número es necesario procesar la salida, calculando el acarreo y asignándolo a la posición correspondiente.

Implementación

## 5.1 Análisis teórico de la complejidad

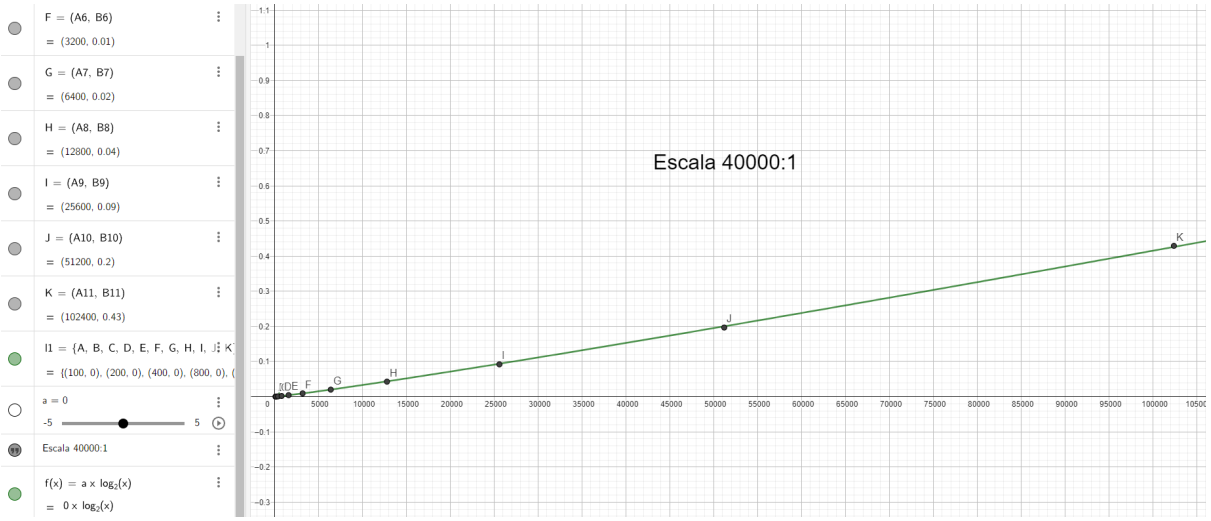
Dado que el algoritmo de la transformada rápida de Fourier tiene una complejidad de O(nlog(n)) y el procesamiento posterior de la salida tiene una complejidad de O(n), la complejidad total del algoritmo es aproximadamente O(nlog(n)).

## 5.2 Análisis experimental de la complejidad

| **Transformada Rápida de Fourier O(nlog(n))** | |
|---|---|
| Talla (N) | Tiempo[s] |
| 100 | 0.00026 |
| 200 | 0.00046 |
| 400 | 0.00094 |
| 800 | 0.00205 |
| 1600 | 0.00446 |
| 3200 | 0.00950 |
| 6400 | 0.02011 |
| 12800 | 0.04282 |
| 25600 | 0.09196 |
| 51200 | 0.19672 |

| | |
|---|---|
| 102400 | 0.42912 |

### 5.3 Gráfica puntos en el plano




[Algoritmos utilizados para la realización del trabajo]

## 6. Bibliografía


- https://github.com/nAuTahn/Fast-Fourier-Transform-FFT/blob/main/FFT%20-%20NTT/FFT.cpp
- https://www.naukri.com/code360/library/karatsuba-algorithm-for-fast-multiplication-of-large-decimal-numbers-represented-as-strings
- http://numbers.computation.free.fr/Constants/Algorithms/fft.html
- https://en.wikipedia.org/wiki/Karatsuba_algorithm